

Workshop on Advanced Sampling Methodologies

Earth Observation data for sampling

Dr. Gianluca Boo, WorldPop, University of Southampton

2025-09-17

Agenda

- Spatial data in R
- Vector and raster data
- Packages: **sf** and **terra**
- Reading and visualising spatial data
- Spatial analysis and visualisation
- Remote sensing with Google Earth Engine

Spatial data in R

- Two main types:
 - **Vector**: points, lines, polygons
 - **Raster**: regular grids with values
- R packages:
 - **sf**: modern vector handling
 - **terra**: raster analysis
- Both integrate well with **ggplot2** for visualisation

Vector data

Raster data

Reading vector data with sf

- **Simple Features Standard:** implements the Simple Features standard for spatial vector data, allowing consistent and interoperable representation of geometries like points, lines, and polygons.
- **Data Frame Integration:** stores spatial data in a regular R data frame (or tibble), with a special geometry column, making it easy to manipulate using familiar R tools like dplyr. Efficient
- **Spatial Operations:** provides a wide range of spatial functions (e.g. intersections, buffers, joins) and interfaces with libraries like GEOS, GDAL, and PROJ for robust, fast, and accurate spatial processing.

```
1 library(sf)
2 admin_boundaries <-
3   "data/pak_adm_wfp_20220909_shp/pak_admbnda_adm1_wfp_20220909.shp" |>
4   st_read(quiet = T)
5
6 admin_boundaries |>
7   st_geometry() |>
8   plot()
```

 Try to run the code in your Script Editor.

Vector data attributes in sf

- **Simple feature collection:** an `sf` object is a data frame-like structure where each row represents a spatial feature (e.g. a region), and includes attribute fields alongside a geometry column.
- **Geometry and dimensions:** each feature has a specific geometry type (e.g. MULTIPOLYGON), with defined spatial dimensions (e.g. XY), used to represent shapes on a map.
- **Spatial metadata:** the object includes spatial metadata such as a coordinate reference system (CRS), e.g. WGS 84, which defines how the geometries relate to real-world locations.

```
1 library(sf); library(tidyverse)
2 admin_boundaries |>
3   select(ADM1_EN) |>
4   head(n=1)
```

Simple feature collection with 1 feature and 1 field

Geometry type: MULTIPOLYGON

Dimension: XY

Bounding box: xmin: 73.39804 ymin: 32.76917 xmax: 74.86488 ymax: 35.13466

Geodetic CRS: WGS 84

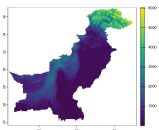
	ADM1_EN	geometry
1	Azad Kashmir	MULTIPOLYGON (((74.46635 35...

 Try to run the code in your Script Editor.

Reading raster data with terra

- **Modern Raster and Vector Support:** `terra` provides tools for reading, writing, analyzing, and manipulating raster and vector spatial data, designed as a faster and more efficient successor to the older `raster` package.
- **High Performance and Scalability:** built for performance, `terra` can handle large spatial datasets efficiently, supports parallel processing, and integrates well with geospatial libraries like GDAL, PROJ, and GEOS.
- **Intuitive Syntax and Consistency:** offers a consistent and user-friendly syntax for spatial operations (e.g., cropping, masking, aggregation, reprojection), making it easy to work with complex geospatial workflows in R.

```
1 library(terra)
2 elevation <-
3   "data/pak_elevation_merit103_10km_v1.tif" |>
4   rast()
5 elevation |>
6 plot()
```



Raster data attributes in terra

- **Spatial Structure:** raster data is represented as a `SpatRaster` object with defined dimensions (rows, columns, layers), resolution (cell size), and extent (bounding coordinates of the spatial area).
- **Coordinate Reference System (CRS):** each raster includes a `CRS` (e.g., WGS 84, EPSG code) that defines how the raster aligns with real-world geographic space.
- **Metadata and Values:** rasters store metadata such as source file, layer names, and value ranges (e.g., min and max), which are critical for interpretation and analysis of the data.

```
1 library(terra)
2 elevation
```

```
class      : SpatRaster
dimensions : 162, 204, 1  (nrow, ncol, nlyr)
resolution : 0.08333333, 0.08333333  (x, y)
extent     : 60.8725, 77.8725, 23.59167, 37.09167  (xmin, xmax, ymin, ymax)
coord. ref.: lon/lat WGS 84 (EPSG:4326)
source     : pak_elevation_merit103_10km_v1.tif
name       : elevation_merit103_100m_v1
min value  :                2.328116
max value  :                6050.907715
```

sf::st_crs() vs terra::crs()

- **SCRS Representation:** in `sf`, `st_crs()` retrieves the CRS of a vector object (e.g., `admin_boundaries`), while in `terra`, `crs()` is used for raster objects (e.g., `elevation`) to access or set their CRS.
- **SCRS Compatibility:** spatial operations between vector and raster data require both to have the same CRS. If they differ, transformation is needed to align them.
- **SCRS Transformation:** Use `st_transform()` (from `sf`) to reproject vector data to match the CRS of another dataset (e.g., `st_transform(crs(elevation))` ensures `admin_boundaries` matches the raster CRS).

```
1 st_crs(admin_boundaries)
2 crs(elevation)
3
4 admin_boundaries <-
5   admin_boundaries |>
6   st_transform(crs(elevation))
```

 Try to run the code in your Script Editor.

Summarising raster values within polygons

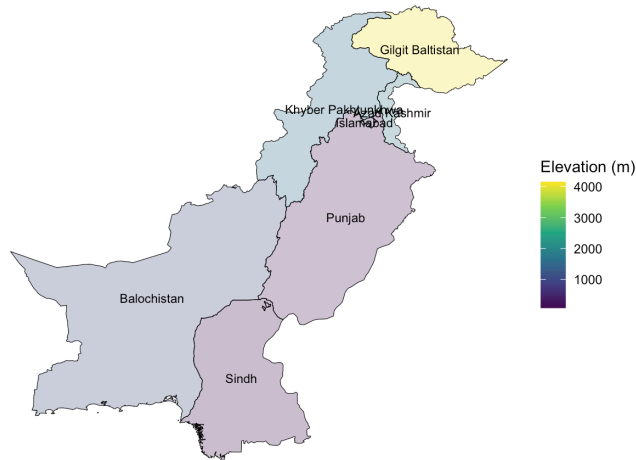
```
1 admin_elevation_mean <-  
2   elevation |>  
3   terra::extract(admin_boundaries, fun = mean, na.rm = TRUE)  
4 admin_boundaries<-  
5   admin_boundaries |>  
6   mutate(elevation_mean=admin_elevation_mean[,2])  
7 admin_boundaries |>  
8   st_drop_geometry() |> dplyr::select(ADM1_EN, elevation_mean)
```

	ADM1_EN	elevation_mean
1	Azad Kashmir	1866.30845
2	Balochistan	983.67310
3	Gilgit Baltistan	4151.73991
4	Islamabad	620.50027
5	Khyber Pakhtunkhwa	1727.04567
6	Punjab	226.31326
7	Sindh	78.63546

Vector map in ggplot2

```
1 library(ggplot2)
2 ggplot() +
3   geom_sf(data = admin_boundaries, aes(fill=elevation_mean), color = "black") +
4   scale_fill_viridis_c()+
5   geom_sf_text(data = admin_boundaries, aes(label = ADM1_EN), size = 3, color = "black") +
6   theme_void()+
7   labs(title="Mean Elevation", fill= "Elevation (m)")
```

Mean Elevation

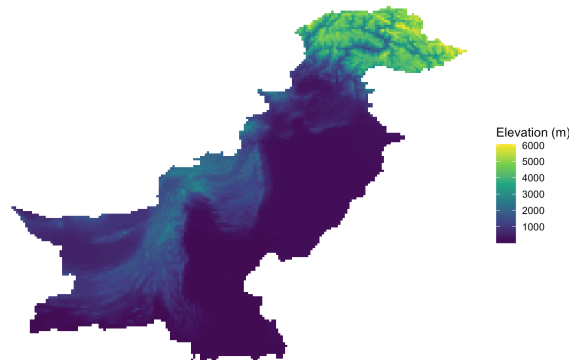


Try to run the code in your Script Editor.

Raster map in ggplot2

```
1 library(ggplot2)
2 elevation_df <- as.data.frame(elevation, xy = TRUE)
3 colnames(elevation_df) <- c("x", "y", "elevation")
4 ggplot(elevation_df) +
5   geom_raster(aes(x = x, y = y, fill = elevation)) +
6   scale_fill_viridis_c() +
7   coord_equal() +
8   theme_void()+
9   labs(title = "Elevation", fill = "Elevation (m)")
```

Elevation



 Try to run the code in your Script Editor.

Google Earth Engine (GEE)

- **Cloud-based platform** for planetary-scale environmental data analysis
- **No need to download massive datasets locally**
- Access to a variety of **pre-processed datasets** including:
 - **Satellite imagery:** Landsat, Sentinel-2, MODIS
 - **Climate data:** ERA5, CHIRPS precipitation
 - **Land cover and vegetation:** Copernicus, GlobCover, NDVI indices
 - **Topography:** SRTM, ASTER DEM
 - **Socioeconomic:** Population density, night-time lights

GEE Code Editor

Open **GEE Code Editor** and follow the following **step-by-step**:

1. Sign in with a Google account (example: `samplingworkshop@gmail.com`)\
2. Click New Script
3. Use the `**search` bar to find datasets
4. Load the dataset with `ee.ImageCollection` or `ee.Image`
5. Use filters (e.g., date, location, bands)
6. Display results with `Map.addLayer()`
7. Export data with `Export.image.toDrive()` or `Export.table.toDrive()`

403. That's an error.

We're sorry, but you do not have access to this page.
That's all we know.

GEE Example Script

```
1 // Example: extract mean NDVI for a region in 2020
2 var roi = ee.Geometry.Rectangle([34.5, -1.5, 35.5, -0.5]);
3 var collection = ee.ImageCollection('MODIS/006/MOD13Q1')
4     .filterBounds(roi)
5     .filterDate('2020-01-01', '2020-12-31')
6     .select('NDVI');
7
8 var mean_ndvi = collection.mean();
9 Map.centerObject(roi);
10 Map.addLayer(mean_ndvi, {min:0, max:9000, palette: ['white','green']}, 'Mean NDVI 2020');
11
12 // Export to Google Drive
13 Export.image.toDrive({
14     image: mean_ndvi,
15     description: 'mean_ndvi_2020',
16     scale: 250,
17     region: roi
18 });
```

 Try to run the code in the GEE Script Editor.

Downloading data from GEE

Use `Export.image.toDrive()` to save a GeoTIFF

Options:

- **Scale:** spatial resolution (meters per pixel)
- **Region:** area of interest (polygon)
- **CRS:** coordinate reference system
- **Format:** GeoTIFF for rasters, CSV/GeoJSON for tables

After export, download the file from Google Drive

 Try to download the file from Google Drive.

Reading GEE data

```
1 library(terra); library(sf); library(ggplot2)
2
3 admin_boundaries <- st_read("data/pak_adm_wfp_20220909_shp/pak_admbnda_adm1
4
5 ndvi_raster <- rast("data/mean_ndvi_2020.tif")
6
7 admin_boundaries <-
8   admin_boundaries |>
9   st_transform(crs(ndvi_raster))
10
11 ndvi_mean <- terra::extract(ndvi_raster, polygons, fun = mean, na.rm = TRUE)
12 admin_boundaries$mean_ndvi <- ndvi_mean[,2]
13
14 ggplot(polygons) +
15   geom_sf(aes(fill = mean_ndvi)) +
16   scale_fill_viridis_c() +
17   ggtitle("Mean NDVI per Polygon from GEE")
```

 Try to run the code in your Script Editor.

Take home

- Spatial data: vector (sf) vs raster (terra)
- Always check CRS before analysis
- Use sf for points, lines, polygons
- Use terra for raster analysis
- Combine rasters and vectors for summaries and plots
- Use GEE to access earth observation data

Resources

Pebesma, E. (2018). *Simple Features for R: Standardized Support for Spatial Vector Data*. The R Journal, 10(1), 439–446. <https://journal.r-project.org/archive/2018/RJ-2018-009/index.html>

Hijmans, R. J. (2023). *terra: Spatial Data Analysis*. R package version 1.7-36. <https://cran.r-project.org/package=terra>

R Core Team. (2025). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria. <https://www.R-project.org>

Gorelick, N., Hancher, M., Dixon, M., Ilyushchenko, S., Thau, D., & Moore, R. (2017). *Google Earth Engine: Planetary-scale geospatial analysis for everyone*. Remote Sensing of Environment, 202, 18–27. <https://developers.google.com/earth-engine>

Grolemund, G., & Wickham, H. (2017). *R for Data Science: Import, Tidy, Transform, Visualize, and Model Data*. O'Reilly Media. <https://r4ds.had.co.nz>

-

Exercise

Please download the R script with exercises from **GitHub** and try to complete it.